
Definição do trabalho da MI – LABIRINTO GIRATÓRIO

Data da entrega: ____ / ____ / ____

Data da defesa: ____ / ____ / ____

Modalidade: em TRIO.

Visão Geral:

O trabalho consiste no desenvolvimento de um jogo de labirinto 2D em que o cenário inteiro pode ser girado 90° para a esquerda ou para a direita. O jogador controla um personagem que precisa atravessar o labirinto até a saída.

Espalhados pelo mapa existem blocos soltos que obedecem à gravidade. Sempre que o cenário gira, aquilo que era uma parede lateral passa a ser chão, e os blocos caem até encontrar um novo apoio. Existem ainda portas que só são sólidas em determinadas orientações do cenário: ao girar, elas somem e reaparecem, derrubando o que estava apoiado sobre elas. Como consequência, cada rotação reorganiza o labirinto: passagens que estavam bloqueadas se abrem e caminhos que existiam se fecham.

A gravidade sempre aponta para baixo da tela — quem muda é o cenário, não a gravidade. A rotação também não é livre: ela só pode ser acionada quando o personagem estiver sobre uma alavanca. O desafio de cada fase, portanto, é descobrir em que ordem alcançar as alavancas e para que lado girar o cenário.

Elementos do mapa:

O cenário é representado por uma matriz quadrada de inteiros. Cada valor identifica um elemento. Os símbolos são apenas uma sugestão de desenho; a equipe pode escolher outros, inclusive usando cores de fundo como no código fornecido pelo professor.

Código	Símbolo	Elemento	Comportamento
0	(espaço)	Vazio	Atravessável. Não sustenta blocos.
1	#	Parede fixa	Sólida. Gira com o cenário, mas nunca cai.
2	@	Jogador	Ocupa uma célula. Gira com o cenário.
3	O	Bloco solto	Sólido. Gira com o cenário e cai até apoiar.
4	A	Alavanca	Atravessável. Gira, não cai. Habilita a rotação.
5	S	Saída	Atravessável. Gira, não cai. Vence ao ser alcançada.
6	= fechada : aberta	Porta tipo A	Sólida nas orientações 0° e 180°. Some nas orientações 90° e 270°.
7	fechada ; aberta	Porta tipo B	Sólida nas orientações 90° e 270°. Some nas orientações 0° e 180°.

Orientação do cenário:

O jogo precisa saber em qual das quatro orientações o cenário se encontra. Convencionou-se que toda fase começa na orientação 0°; girar para a direita avança 90° e girar para a esquerda retrocede 90°, sempre voltando a 0° depois de quatro giros no mesmo sentido.

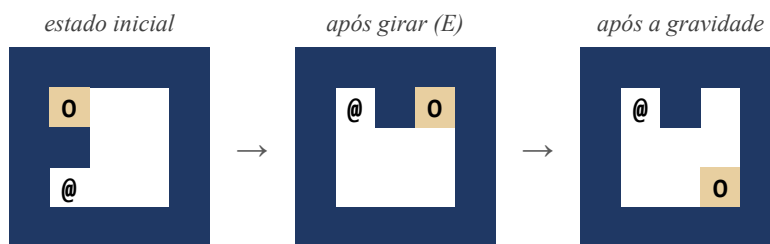
A orientação atual é o que determina quais portas estão sólidas e quais sumiram. Ela deve ser exibida na tela durante a partida e precisa ser preservada pela opção “Continuar” do menu.

Regras do jogo:

1. O jogador se move com as teclas W, A, S e D, uma célula por vez. Ele não atravessa paredes, blocos nem portas fechadas.
2. Estando sobre uma alavanca, o jogador pode pressionar Q para girar o cenário no sentido anti-horário e E para girá-lo no sentido horário. Fora da alavanca, essas teclas não têm efeito.
3. A rotação gira TODO o cenário: paredes, blocos, alavancas, portas, saída e o próprio jogador. Como a rotação é uma transformação célula a célula, nenhum elemento é perdido nem sobreposto nesse momento.
4. Após girar, a nova orientação define o estado das portas. Uma porta fechada bloqueia a passagem e sustenta blocos; uma porta aberta é atravessável e não sustenta nada. A porta continua existindo na matriz nos dois casos — o que muda é apenas a forma como ela é interpretada e desenhada.
5. Em seguida aplica-se a gravidade: todo bloco solto desce enquanto a célula logo abaixo dele estiver vazia, parando ao encontrar uma parede, uma porta fechada, outro bloco ou a borda do mapa.
6. Se uma porta se fechar exatamente sobre a célula ocupada por um bloco, o bloco é esmagado e desaparece do cenário.
7. Se uma porta se fechar exatamente sobre a célula ocupada pelo jogador, a fase é perdida e deve ser reiniciada.
8. O jogador não é afetado pela gravidade — ele caminha livremente pelo labirinto.
9. A partida termina em vitória quando o jogador alcança a saída.
10. A tecla R reinicia a fase e a tecla ESC retorna ao menu. O reinício é obrigatório, pois uma sequência ruim de rotações pode deixar a fase sem solução.
11. A tela deve exibir, durante o jogo, o número do mapa, a orientação atual, a quantidade de movimentos e a quantidade de rotações já realizadas.

Exemplo 1 – rotação e queda de blocos:

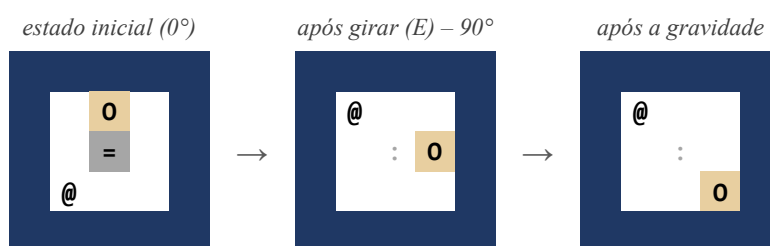
No cenário à esquerda o jogador (@) está sobre a alavanca e o bloco (O) está apoiado na parede logo abaixo dele. O jogador pressiona E e todo o cenário gira; em seguida a gravidade derruba o bloco até a borda inferior.



Observe o efeito: a passagem que ficava à direita do jogador foi fechada por uma parede, enquanto a célula antes ocupada pelo bloco ficou livre. O jogador continua sobre a alavanca e pode girar novamente.

Exemplo 2 – porta que some com a rotação:

Agora o bloco está apoiado sobre uma porta do tipo A (=), sólida enquanto o cenário está a 0°. Ao girar para a direita, o cenário passa para 90° e essa porta some (:), deixando de sustentar o bloco, que cai e abre a passagem que ele ocupava.



Se o jogador girar mais uma vez, o cenário chega a 180° e essa mesma porta volta a ser sólida. Se, nesse momento, houver um bloco parado na célula dela, o bloco é esmagado; se houver o jogador, a fase é perdida.

Regras para o desenvolvimento:

- O jogo deverá ser desenvolvido em C/C++ sem utilizar motores de jogos.
- Devem ser utilizados como ponto de partida os códigos de desenho e movimentação fornecidos pelo professor (versão Linux e versão Windows). A equipe deve informar em qual das duas versões o trabalho foi desenvolvido.
- Não é permitida a utilização de variáveis globais.
- Devem ser utilizados apenas os recursos já vistos na disciplina: funções, passagem de parâmetros por valor e por referência, vetores e matrizes. Não utilizar classes nem estruturas (struct).
- O cenário deve ser uma matriz quadrada de, no mínimo, 11 x 11 posições. Recomenda-se que a borda do mapa seja formada inteiramente por paredes fixas.
- A função main deve ser curta: essencialmente o menu e o laço principal chamando sub-rotinas. Nenhuma regra do jogo deve estar escrita dentro dela.
- Nenhuma sub-rotina deve, ao mesmo tempo, calcular e imprimir. Separe as funções que alteram o cenário das funções que desenhavam o cenário.
- As portas abertas devem continuar visíveis na tela, desenhadas de forma diferente das portas fechadas, para que o jogador consiga planejar as rotações.
- O jogo deverá possuir no mínimo 3 mapas, independentemente do número de integrantes na equipe.
- Os mapas são autorais e deverão ser submetidos ao professor antes da implementação, que garantirá que nenhuma equipe implemente um mapa semelhante ao de outra.
- Junto com cada mapa, a equipe deve entregar uma sequência de teclas que o resolve. Um mapa sem solução comprovada não será considerado.

- Entre os mapas entregues deve haver, obrigatoriamente:
 - o pelo menos um em que a solução dependa da queda de um bloco para abrir uma passagem;
 - o pelo menos um em que a solução dependa de uma porta que some com a rotação;
 - o pelo menos um que exija duas ou mais rotações para ser concluído.
- O estado inicial de todo mapa deve estar estável, ou seja, nenhum bloco pode começar a fase no ar.

Menu:

O jogo deverá possuir um menu onde será possível escolher: Novo jogo, Continuar, Sobre e Fim. A seguir são listadas, de trás para frente, as exigências de cada opção.

Fim:

Essa é a opção para finalizar o programa. Observe que seu jogo só deve ser encerrado ao selecionar essa opção; qualquer outra opção deve retornar ao menu ao fim de sua execução. Deve ser possível retornar ao menu a qualquer momento.

Sobre:

Quando essa opção for selecionada, deverão ser exibidos a equipe de desenvolvimento (o nome de cada membro), o mês/ano e o nome do professor/disciplina. Além disso, essa opção deverá explicar as regras do jogo e a função de cada tecla.

Continuar:

Quando essa opção for selecionada, o jogador deverá continuar de onde parou, com o cenário exatamente na orientação em que foi deixado. Essa opção só deverá estar disponível caso o jogador já tenha um jogo em andamento.

Novo Jogo:

Quando essa opção for escolhida, o jogador terá ainda que decidir se deseja jogar em um mapa específico ou iniciar em um mapa aleatório.

Sobre o uso de sub-rotinas:

Este é o principal critério técnico do trabalho. Espera-se que o cenário, a orientação e as posições relevantes circulem entre as funções como parâmetros, e não como variáveis visíveis a todo o programa. Lembre-se de que uma função não devolve uma matriz pelo retorno: a matriz de destino também precisa ser um parâmetro.

A decomposição abaixo é um mínimo esperado. A equipe pode (e deve) criar outras funções, mas não deve concentrar responsabilidades em funções gigantes.

- ler uma tecla do teclado;
- desenhar o cenário na tela, considerando a orientação atual;
- informar se uma porta está fechada em uma dada orientação;
- informar se uma determinada célula pode ser atravessada;
- informar se uma determinada célula sustenta um bloco;
- mover o jogador em uma direção;

- girar o cenário para a direita e girar o cenário para a esquerda;
- aplicar a gravidade sobre os blocos;
- resolver o esmagamento causado pelas portas que se fecharam;
- localizar o jogador dentro da matriz;
- informar se o jogador está sobre uma alavanca;
- verificar as condições de vitória e de derrota;
- carregar um mapa a partir do seu número;
- exibir o menu e exibir a tela “Sobre”.

Exemplos de assinatura, apenas para ilustrar a forma esperada:

```
void desenhaCenario(int m[][TAM], int n, int orientacao);
bool portaEstaFechada(int celula, int orientacao);
void giraDireita(int origem[][TAM], int destino[][TAM], int n);
void aplicaGravidade(int m[][TAM], int n, int orientacao);
void moveJogador(int m[][TAM], int n, int &px, int &py, int orientacao, char
tecla);
```

Desafios de projeto:

Alguns pontos não têm resposta única e fazem parte da avaliação. A equipe deve decidir e justificar suas escolhas na apresentação:

- Como representar duas coisas na mesma célula, por exemplo o jogador parado sobre a alavanca, um bloco parado sobre a saída ou o jogador dentro de uma porta aberta.
- Em que ordem varrer a matriz ao aplicar a gravidade, de modo que uma pilha de blocos caia corretamente de uma só vez.
- Em que momento resolver o esmagamento das portas: antes ou depois de aplicar a gravidade, e por quê.
- Como garantir que a matriz auxiliar usada na rotação não corrompa o cenário original.
- Como tratar um mapa que ficou sem solução após uma rotação.

OBSERVAÇÃO

Esse jogo tem por objetivo avaliar a utilização de técnicas de programação aprendidas na disciplina, principalmente quanto ao uso de sub-rotinas e à passagem de parâmetros. Além disso, produzam todo o código pensando na possibilidade de que novas funcionalidades poderão ser solicitadas no futuro.

ENTREGA:

- Postar um arquivo texto com o link para o trabalho no GitHub.
- Anexar os mapas e a respectiva sequência de teclas que resolve cada um deles.

APRESENTAÇÃO:

- A apresentação e a nota da apresentação serão individuais; além disso, todas as notas relativas ao código dependem do desempenho na apresentação. Sem apresentação, o trabalho terá nota ZERO.

Crítérios de Avaliação:

1. Organização e clareza do código = 10% da nota.
2. Funcionamento correto conforme a especificação = 20% da nota.
3. Recursos da linguagem utilizados, com destaque para o uso de sub-rotinas e passagem de parâmetros = 20% da nota.
4. Apresentação do código = 50% da nota e é individual.

Obs1: Cópia de soluções de colegas ou da internet acarretarão nota 0 para todos os envolvidos.

Obs2: Trabalhos entregues e/ou apresentados em atraso receberão um desconto inicial de 20% na nota final, além de um desconto progressivo de 10% por aula. Ou seja, apresentar na aula seguinte à data estipulada deixa a nota máxima do trabalho em 70%.